Ayush Sharma

MSE800_Assessment_Report

Assessment-1

Document Details

Submission ID
trn:oid:::3618:151544099

Submission Date
Sep 13, 2026, 12:48 PM GMT+12

Download Date
Sep 13, 2026, 12:49 PM GMT+12

File Name
MSE800_Assessment_Report.pdf

File Size
460.5 KB

6 Pages

549 Words

3,274 Characters

2% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

1 Not Cited or Quoted 2%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

2% Internet sources
0% Publications
0% Submitted works (Student Papers)

Match Groups

- **1** Not Cited or Quoted 2%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

2% Internet sources
0% Publications
0% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1 Internet

medium.com **2%**



Car Rental System

MSE800 Professional Software Engineering
Assignment 1

Supervisor:

Mohammad Norouzifard

Student Name:
Ayush Sharma

Student ID:
270929483

Institution:
Yoobee College of Creative Innovation

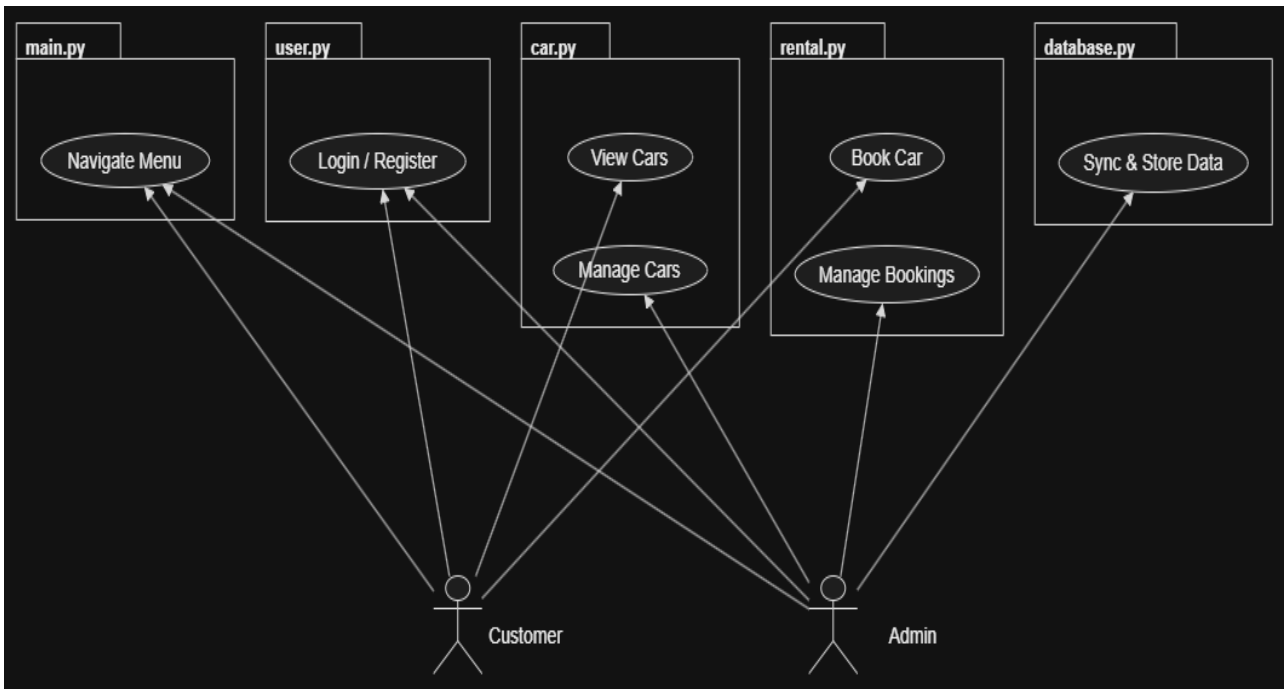
Date of Submission:
13 September 2026

Task 1: Design and Architecture

The Car Rental System uses a modular, object-oriented architecture in Python. The design separates concerns into logical components: User Session management, Core Business Logic, Data Storage, and the Application UI. This structure enhances maintainability and scalability.

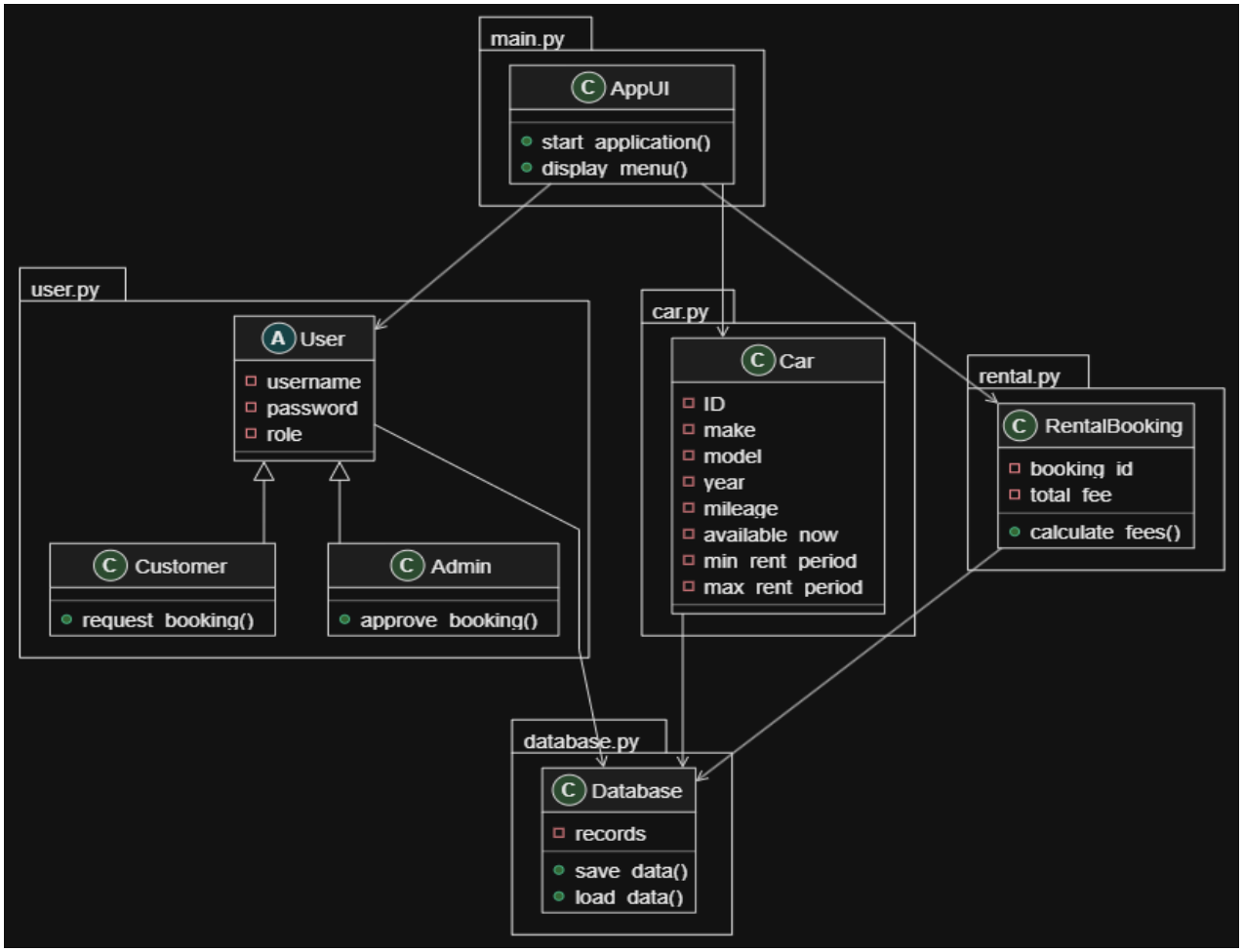
Use Case Diagram

This diagram maps out the core user and car management functionalities while defining the specific boundaries for both Customer and Admin roles.



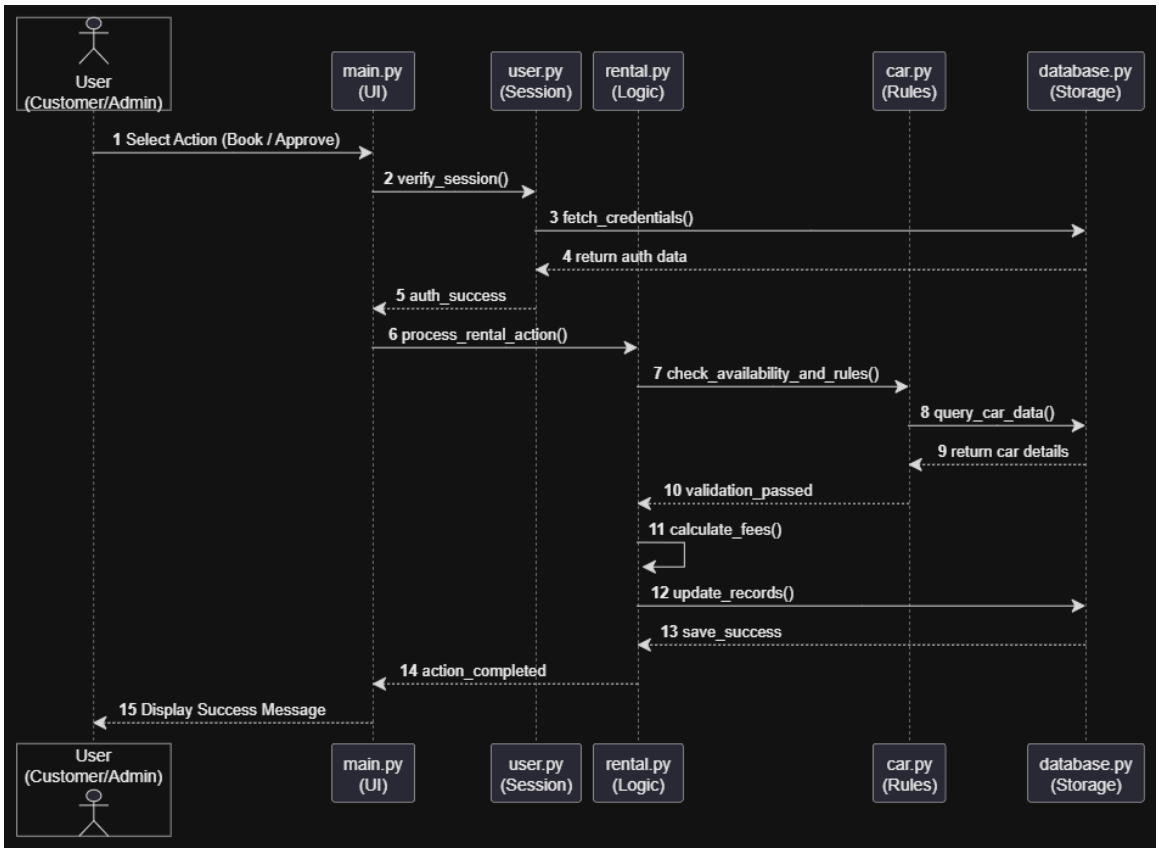
Class Diagram

This architecture fulfils the requirement to utilise proper design structures. It includes a database storage approach that retains the exact variables required for car records (e.g., mileage, availability, and minimum/maximum rental periods).



Sequence Diagram

This sequence illustrates the Rental Booking and Rental Management flow. It tracks the exact process of a customer selecting dates, the system calculating rental fees, and the Admin approving or rejecting the booking.



Task 2: Innovative Solutions

Innovative Feature: IoT Mobile Key & Telematics Integration

To set this car rental system apart from traditional manual paperwork processes, the architecture incorporates an "IoT Mobile Key Unlock" feature. Traditional rental agencies require customers to physically visit a desk during business hours to retrieve keys, creating bottlenecks and limiting operational hours.

This enhancement integrates the rental software with IoT telematic devices installed in the fleet. Once an Admin changes a booking status to "Approved" in the system, it generates a secure, time-sensitive digital token. The system transmits this token to a companion mobile application, allowing the customer to unlock the car via Bluetooth or NFC directly from their smartphone.

Industry Requirement & Competitive Advantage

This addresses a critical industry shift toward contactless, self-service rentals. It provides a significant competitive advantage by enabling 24/7 vehicle dispatch without requiring round-the-clock on-site staffing. Furthermore, because the system tracks precise variables like mileage and available_now, future iterations of this IoT integration can automatically sync odometer readings back to the database at the end of the rental period, completely automating the check-in and billing process.

Task 3: Software Evolution

Software Maintenance Strategy

Ongoing maintenance will follow a strict Git-based workflow. All new features or bug fixes (such as implementing UI input validation) will be developed on isolated feature branches before being merged into the main repository. Routine audits of the local database read/write operations will be conducted as the file grows to ensure performance considerations are met.

Versioning

1

The system will utilise Semantic Versioning (SemVer) following a MAJOR.MINOR.PATCH format.

- **Patch (1.0.1):** Used for bug fixes, such as resolving terminal crash edge cases on invalid inputs.
- **Minor (1.1.0):** Used for adding functionality in a backwards-compatible manner, such as adding a new attribute to the booking system.
- **Major (2.0.0):** Used for significant architectural shifts, such as migrating from a local JSON file to a cloud-based SQL database.

Backward Compatibility

To support long-term success, we will prioritise backward compatibility during data schema updates. If we add new fields to the data models in future versions, we will update the database loading module with default fallback values (e.g., using dictionary .get() methods with default values) to prevent the system from crashing when reading older serialised records.